

CardzCheck App Summary (Repo-Based)

Evidence sources: app/, components/, lib/, supabase/migrations/, QUICK_START.md, .env.example

What It Is

CardzCheck is a Next.js sports card platform that combines comps search, estimated CMV, collection tracking, and AI-assisted workflows in one app. It integrates pricing, card identification, grading analysis, and subscription controls through a single authenticated experience.

Who It Is For

Primary persona: sports card collectors and hobby investors/resellers who need faster pricing checks, collection visibility, and decision support.

Detailed formal persona document: Not found in repo.

What It Does

- Runs card comps search with an eBay dual-signal engine and returns CMV-oriented stats (for-sale + estimated sale range).
- Identifies card details from uploaded images (URL or base64) with image validation and Anthropic-powered OCR/vision extraction.
- Lets users add, view, filter, sort, export/import, and maintain collection items with CMV state and card image support.
- Provides a Pro watchlist with target-price tracking and periodic price refresh endpoints.
- Runs grade-estimate jobs from card photos, then computes post-grading value and worth-grading outcomes.
- Offers a Pro analyst chat plus persisted analyst threads/messages with usage controls.
- Supports search assist via player/set typeahead and card-catalog lookup endpoints.

How It Works (Architecture)

- Frontend: Next.js App Router pages (for example /dashboard, /comps, /collection, /watchlist, /grade-estimator) with React client components and AuthContext.
- Backend-for-frontend: Route handlers in app/api/* validate input, enforce auth/plan limits, and orchestrate feature logic.
- Core services in lib/: ebay/* pricing pipeline, card-identity/* extraction, grading/* job flow, stripe.ts billing helpers, and utilities for CMV/search/state.
- Data layer: Supabase auth + Postgres tables (users, collection_items, card_images, watchlist, recent_searches, analyst_threads, grade_estimator_runs, subscriptions, usage) inferred from routes/types/migrations.
- Request flow: Browser UI -> /api routes -> lib services -> external providers (Supabase, eBay, Anthropic, Stripe) -> JSON -> UI state updates.

How To Run (Minimal)

- Copy env template and set keys: cp .env.example .env, then fill Supabase, Anthropic, and Stripe variables listed in .env.example.
- Create Supabase Storage bucket card-images (public) and apply bucket policies documented in SUPABASE_STORAGE_SETUP.md.
- Start app: npm run dev, then open http://localhost:3000.
- Local DB migration/seed command for full local stack: Not found in repo.